S09 Events

CRE132 worksheet

What you'll make

S01's Health script learns to **announce** what happens to it. Other things listen and react: a hurt sound in the fire, a heal sound from a potion, a thud when you hit a dummy, a **You Died** panel, and a health bar that only updates when the health changes.

By the end you'll know two kinds of event: **UnityEvents**, which you wire up in the Inspector with no code, and **C# events**, which scripts subscribe to in code. Health never needs to know who is listening.

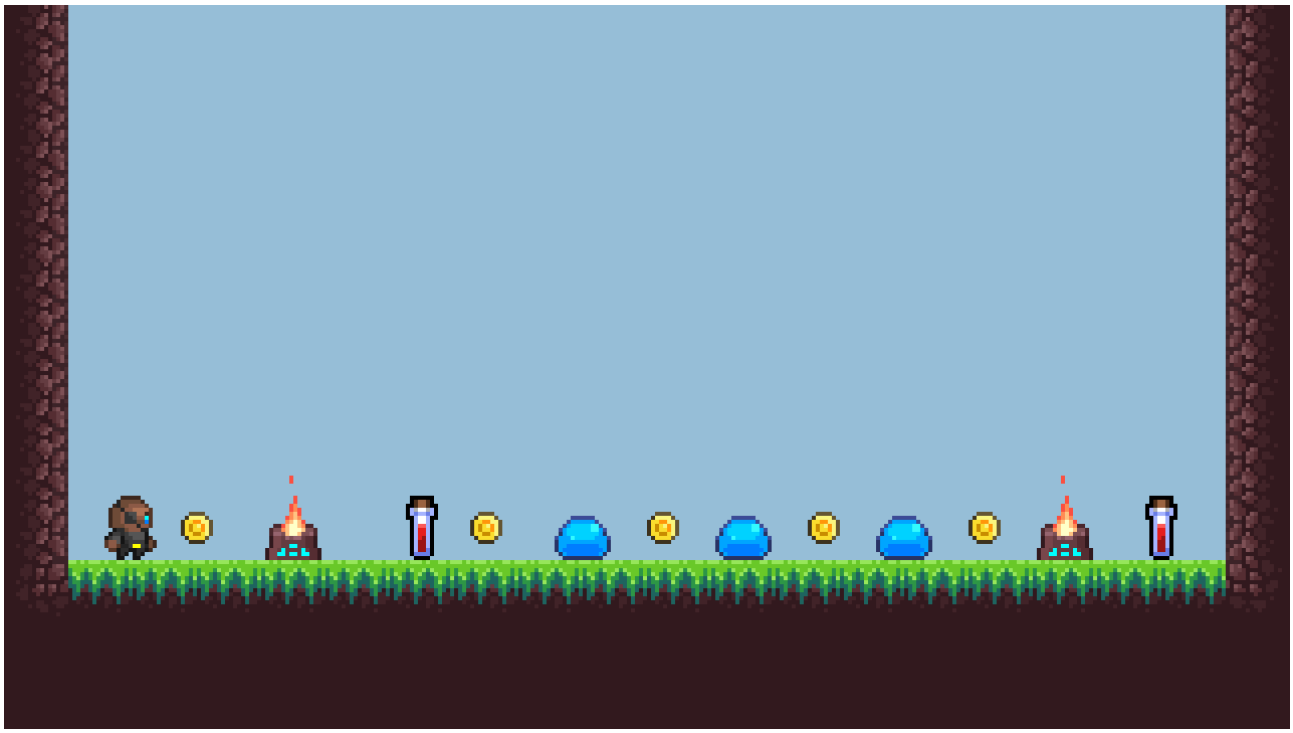


Figure 1: The finished scene: the player, coins, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S09_Events_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S09_Events_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S09_Events`.
3. Press **Play**. It's S03's room: the fire hurts, potions heal, J attacks and coins score, but it's all silent, the health bar never moves, and dying just leaves an empty level. Press Play again to stop.

The scene has a **Sound Effects** object with an **Audio Source**, and a hidden **Death Panel** on the HUD with a **Try Again** button. The sounds are in `Assets/_game/Audio/SFX`.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Health announces things: UnityEvents

S03's HealthBar checks the health every frame, and S05 needed a whole HurtSound script that checks too. With events, Health announces "I've been damaged", and anything interested reacts.

Open `Health` (`Assets/_game/Scripts/Health`). Add this line below `using UnityEngine;`:

```
using UnityEngine.Events;
```

Add three events below the `maxHealth` line:

```
    // UnityEvents: lists of things to do, filled in the Inspector (play a sound, show
    // a panel...).
    [SerializeField] private UnityEvent onDamaged = new UnityEvent();
    [SerializeField] private UnityEvent onHealed = new UnityEvent();
    [SerializeField] private UnityEvent onDied = new UnityEvent();
```

In `TakeDamage`, add this line below the `Debug.Log(... " took " ...)` line:

```
        onDamaged.Invoke();
```

In `Heal`, add this line below its `Debug.Log` line:

```
        onHealed.Invoke();
```

And in `Die`, add `onDied.Invoke();` before the object is destroyed, so `Die` looks like this:

```
    private void Die()
    {
        Debug.Log(gameObject.name + " died");
        onDied.Invoke();
        Destroy(gameObject);
    }
```

- **A `UnityEvent`** is a list of things to do. Because it's `[SerializeField]`, the list shows in the Inspector and you fill it there, like a Button's **On Click ()**.
- **`Invoke()`** does everything in the list. Health doesn't know or care what's in it.
- **`onDied` comes before `Destroy`**, so the listeners run while the object still exists.

2. Wiring events in the Inspector

Select the **Player**. Its Health component now shows **On Damaged ()**, **On Healed ()** and **On Died ()**. For each one, click **+**, drag an object into the slot, and pick a function from the dropdown:

- **On Damaged ():** drag in **Sound Effects**, pick **AudioSource > PlayOneShot (AudioClip)**, and put `GS1_Damage_1` in the clip slot below it.
- **On Healed ():** the same, with `GS2_Item_Acquire_1`.
- **On Died ():** drag in **Death Panel**, pick **GameObject > SetActive (bool)**, and tick the box.

Then select all three **Training Dummies** and give their **On Damaged ()** the same **Sound Effects > PlayOneShot**, with `GS1_Hit_1`.

- **Why a separate Sound Effects object?** The dummy is destroyed on its last hit; an Audio Source on the dummy would be destroyed too and cut the sound off. Sound Effects stays in the scene.
- **No new scripts** for any of this: the sounds and the panel are set up on each Health component.

Test it: stand in the fire (ouch), drink a potion (ping), hit a dummy with J (thud). Stay in the fire until you die: **You Died** appears, and **Try Again** restarts the level.

3. A C# event for scripts

UnityEvents are great for wiring things up in the Inspector. When the listener is a script, a **C# event** is simpler and faster. Back in `Health`, add this below the three UnityEvents:

```
    // A C# event: other scripts subscribe to it in code (health.HealthChanged +=
    // MyMethod).
    // The two numbers sent are the current and maximum health.
    public event System.Action<int, int> HealthChanged;
```

Add this method at the end of the class, below `Die`:

```
    private void SendHealthChanged()
    {
        // Nobody may be listening: an event with no subscribers is null.
        if (HealthChanged != null)
        {
            HealthChanged(currentHealth, maxHealth);
        }
    }
```

Now call it whenever the health changes. Below `onDamaged.Invoke();` in `TakeDamage`, and below `onHealed.Invoke();` in `Heal`, add:

```
        SendHealthChanged();
```

Also add it to `SetHealth` (from S08), so it looks like this:

```
    public void SetHealth(int amount)
    {
        currentHealth = Mathf.Clamp(amount, 1, maxHealth);
        SendHealthChanged();
    }
```

- **`System.Action<int, int>`** means "a method that takes two ints". Anyone who subscribes must give a method of that shape.
- **`HealthChanged(currentHealth, maxHealth)`** calls every subscribed method with the two numbers.
- **The `null` check:** an event that no one has subscribed to is `null`, and calling it would throw an error.

4. A health bar that listens: EventHealthBar

Create `EventHealthBar` in `Assets/_game/Scripts/UI`:

```
// S03's HealthBar rewritten with an event: instead of checking the Health every
// frame, it subscribes to Health.HealthChanged and only runs when the health actually
// changes.
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class EventHealthBar : MonoBehaviour
{
```

```
    [SerializeField] private Health health;
    [SerializeField] private Image fillImage;
    [SerializeField] private TMP_Text healthText;

    // Subscribe when enabled, unsubscribe when disabled. Forgetting to unsubscribe
    // leaves the Health calling a method on a destroyed object.
    private void OnEnable()
    {
        health.HealthChanged += UpdateBar;
    }

    private void OnDisable()
    {
        if (health != null)
        {
            health.HealthChanged -= UpdateBar;
        }
    }

    private void Start()
    {
        // The event only fires on a change, so show the starting health ourselves.
        UpdateBar(health.CurrentHealth, health.MaxHealth);
    }

    private void UpdateBar(int currentHealth, int maxHealth)
    {
        fillImage.fillAmount = (float)currentHealth / maxHealth;
        healthText.text = currentHealth + " / " + maxHealth;
    }
}
```

- **`+=` subscribes** `UpdateBar` to the event; **`-=` unsubscribes**. Always unsubscribe in `OnDisable` (or `OnDestroy`), or a destroyed listener can still be called.
- **`UpdateBar(int currentHealth, int maxHealth)`** matches the event's shape: two ints.
- **No `Update` at all.** Compare it with S03's HealthBar: same result, but this one does nothing until something happens.
- **`Start` calls `UpdateBar` once** because the event only fires on a change, and nothing has changed yet.

Add it to **Health Bar**: drag the **Player** into **Health**, **Fill** into **Fill Image** and **Health Text** into **Health Text**.

Test it: stand in the fire. The bar drops with each hit, and grows when you drink a potion.

Check your work

- ☐ The fire plays a hurt sound, a potion a heal sound, and hitting a dummy a thud, even on its last hit.
- ☐ Dying shows **You Died**, and Try Again restarts.
- ☐ The health bar follows the player's health.
- ☐ Compare with `CRE132_S09_Events_Finished` if something behaves differently.

Key points

- **UnityEvents** are wired in the Inspector: great for one-off reactions (sounds, panels, animations), set up without code.
- **C# events** are subscribed to in code: better when the listener is a script you own, and faster.

- **Always unsubscribe** (`-=`) in `OnDisable` or `OnDestroy`.
- **The object raising the event never needs a reference to its listeners.** That's what keeps Health reusable.

If something goes wrong

- **"The type or namespace name 'UnityEvent' could not be found":** the `using UnityEngine.Events;` line is missing.
- **The On Damaged () list doesn't show in the Inspector:** Health has an error, or the events aren't `[SerializeField]`. Fix the first red error in the Console.
- **No sound:** the event's object slot is empty, the function is still **No Function**, or the clip slot under it is empty.
- **You Died appears at the start:** the **Death Panel** must stay switched off in the Hierarchy; only On Died switches it on.
- **NullReferenceException in EventHealthBar's OnEnable:** the **Health** slot is empty.
- **The bar doesn't move:** `SendHealthChanged();` is missing from `TakeDamage` or `Heal`.

Try it

1. Add a `ScoreChanged` event to the GameManager and make an event-driven score display.
2. Use a dummy's **On Died ()** to make a coin appear. (`Instantiate` needs code, so place a hidden coin and use **GameObject > SetActive** instead.)
3. An event list can hold as many things as you like. Give the player's **On Damaged ()** a second entry as well as the sound. (In E02 and E04 you'll add a camera shake and a white flash here.)